

UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingeniería de Sistemas
Informáticos



UNIVERSIDAD
POLITÉCNICA
DE MADRID



Máster
Deep Learning
Universidad Politécnica de Madrid

Trabajo Fin de Máster

Sistema LLM para análisis financiero histórico
mediante generación automática de código

MÁSTER DE FORMACIÓN PERMANENTE EN DEEP LEARNING

Presentado por:

Carlos Ruiz Oyarzun

Bajo la dirección de:

Adrián Girón Jiménez

Alejandro Delgado Peribáñez

Madrid, 2026

Resumen

Este trabajo presenta el diseño, implementación y evaluación de un sistema multi-agente basado en modelos de lenguaje para realizar análisis financiero histórico a partir de consultas en lenguaje natural. El objetivo del proyecto no es predecir el mercado ni recomendar inversiones, sino estudiar si un LLM puede integrarse en un flujo más controlado que transforme una petición abierta en una respuesta sustentada por datos y cálculos observables.

El sistema desarrollado organiza el proceso en varias etapas: resolución y validación de datos históricos, planificación del análisis, generación de código Python, validación previa a la ejecución, ejecución controlada e interpretación final. Como aportación principal, la arquitectura incorpora trazabilidad completa y mecanismos de control que permiten conservar artefactos intermedios, detectar fallos por fase y acotar las reparaciones cuando aparecen errores. La evaluación se apoya en pruebas automatizadas y en una batería de consultas financieras de distinta dificultad. En conjunto, los resultados muestran la viabilidad de una arquitectura LLM más explicable, revisable y robusta que una respuesta generativa directa de una sola etapa.

Palabras clave: análisis financiero histórico, modelos de lenguaje, generación de código, agentes LLM, ejecución controlada, trazabilidad.

Abstract

This dissertation presents the design, implementation and evaluation of a multi-agent language-model-based system for historical financial analysis from natural-language queries. The aim is not to predict market behaviour or provide investment advice, but to study whether an LLM can be integrated into a controlled workflow that turns an open request into a response grounded in observable data and calculations.

The implemented system structures the process into several stages: historical data resolution and validation, analysis planning, Python code generation, pre-execution validation, controlled execution and final interpretation. Its main contribution lies in the combination of full traceability and explicit control mechanisms, which make it possible to preserve intermediate artefacts, identify failures by stage and bound repair attempts when errors occur. The evaluation relies on automated tests and on a graded set of financial-analysis queries with different levels of difficulty. Overall, the results support the feasibility of an LLM-based architecture that is more explainable, reviewable and robust than a single-step direct generative response.

Keywords:

historical financial analysis, language models, code generation, LLM agents, traceability.

Índice general

Resumen	I
Abstract	II
Abreviaturas y acrónimos	VIII
1. Introducción	1
1.1. Contexto y definición del problema	1
1.2. Motivación	2
1.3. Objetivos	2
1.4. Alcance	3
1.5. Estructura de la memoria	3
2. Estado de la cuestión	5
2.1. Enfoque de la revisión	5
2.2. Modelos de lenguaje y agentes financieros	5
2.3. Agentes, herramientas y orquestación	6
2.4. Generación y ejecución controlada de código	7
2.5. Fiabilidad y evaluación de sistemas generativos	7
2.6. Comparativa y posicionamiento de la propuesta	8
3. Material de trabajo y planteamiento experimental	10
3.1. Origen y naturaleza del material de trabajo	10
3.2. Casos y escenario experimental de partida	11
4. Metodología	13
4.1. Funcion de este capítulo	13
4.2. Enfoque metodológico	13
4.3. Preguntas guía para escribir bien la metodología	13
4.3.1. Sobre el objetivo de evaluación	13
4.3.2. Sobre la unidad de análisis	14
4.3.3. Sobre el diseño experimental	14

4.3.4.	Sobre la evaluacion	14
4.3.5.	Sobre la validez del estudio	14
4.4.	Estructura recomendada para la metodologia final	15
4.4.1.	Pregunta de investigacion y objetivos evaluables	15
4.4.2.	Unidad de analisis	15
4.4.3.	Diseno experimental	15
4.4.4.	Protocolo de ejecucion	15
4.4.5.	Metricas de evaluacion	15
4.4.6.	Revision humana	15
4.4.7.	Amenazas a la validez	16
4.5.	Criterio de calidad para este capitulo	16
5.	Flujo multiagente del sistema	17
5.1.	Funcion de este capitulo	17
5.2.	Preguntas que debe responder el capitulo	17
5.3.	Vision general del flujo	18
5.4.	Entrada y estado compartido	19
5.4.1.	Entrada del sistema	19
5.4.2.	Estado compartido	20
5.5.	Flujo principal del sistema	21
5.5.1.	Parte de datos	21
5.5.2.	Analista y generador de codigo	22
5.5.3.	Validacion del codigo	23
5.5.4.	Validacion de ejecucion	24
5.5.5.	Interpretacion y respuesta final	26
5.6.	Subagentes de correccion y recuperacion	27
5.7.	Diseno de prompts y contratos de los agentes	28
5.8.	Trazabilidad y artefactos generados	28
5.9.	Decisiones de diseno	30
6.	Resultados y evaluacion	31
6.1.	Funcion de este capitulo	31
6.2.	Enfoque recomendado	31
6.3.	Preguntas que convendra responder mas adelante	31
6.4.	Plantilla de estructura futura	32
6.4.1.	Resultado global	32
6.4.2.	Lectura por grupos de casos	32
6.4.3.	Analisis por etapas	32
6.4.4.	Casos representativos	32
6.4.5.	Discusion critica	32

7. Limitaciones, aspectos eticos y reproducibilidad	33
7.1. Funcion de este capitulo	33
7.2. Limitaciones del estudio	33
7.3. Aspectos eticos y uso responsable	33
7.4. Reproducibilidad	33
7.5. Preguntas de control	34
8. Conclusiones y trabajo futuro	35
8.1. Funcion de este capitulo	35
8.2. Guia para las conclusiones	35
8.3. Guia para el trabajo futuro	35
A. Anexos	39
A.1. Ejecucion del sistema	39
A.1.1. Preparacion minima	39
A.1.2. Ejecucion con ejemplos integrados	39
A.1.3. Ejecucion con una entrada JSON propia	40
A.1.4. Salida y artefactos generados	40
A.2. Configuracion LLM y API	41
A.2.1. Configuracion recomendada	41
A.2.2. Variables equivalentes admitidas	41
A.2.3. Parametros de generacion	42
A.2.4. Comprobaciones practicas	42
A.3. Notas para reproducibilidad	42

Índice de tablas

2.1. Comparación entre los trabajos relacionados y la propuesta desarrollada.	8
4.1. Metricas previstas para la evaluacion del sistema.	15
5.1. Guia para documentar los prompts y contratos de los agentes.	28

Índice de figuras

5.1. Flujo multiagente de referencia para el desarrollo del sistema de analisis financiero.	18
--	----

Abreviaturas y acrónimos

API Application Programming Interface.

CSV Comma-Separated Values.

ETF Exchange-Traded Fund.

LLM Large Language Model.

OHLCV Open, High, Low, Close, Adjusted Close y Volume.

RAG Retrieval-Augmented Generation.

UTC Coordinated Universal Time.

Capítulo 1

Introducción

1.1. Contexto y definición del problema

Los modelos de lenguaje de gran escala han ampliado de forma notable las posibilidades de interacción con los sistemas informáticos. Además de producir texto, pueden interpretar instrucciones, proponer procedimientos y generar código ejecutable. Buena parte de este avance se apoya en la arquitectura Transformer y en el mecanismo de atención, que constituyen una de las bases de los modelos de lenguaje actuales.

La capacidad de generar código resulta especialmente interesante en tareas de análisis de datos. Una consulta expresada en lenguaje natural puede transformarse en operaciones de carga, filtrado, cálculo y presentación de resultados. Sin embargo, esta flexibilidad introduce un problema de confianza: que el código sea plausible no significa que sea correcto, y que una respuesta esté bien redactada no garantiza que respete los datos de partida.

Este problema cobra especial importancia en el ámbito financiero. Incluso cuando el análisis se limita a información histórica, una cifra incorrecta o una interpretación poco prudente puede conducir a conclusiones equivocadas. Por ello, no basta con pedir al modelo que responda directamente a una consulta: también es necesario conocer qué datos ha utilizado, qué métricas ha calculado, qué código ha ejecutado y qué limitaciones presenta el resultado.

Este trabajo aborda el problema desde la integración de sistemas, no desde el entrenamiento de un nuevo modelo. Para ello, se desarrolla un prototipo que recibe una consulta financiera en lenguaje natural, resuelve y descarga los datos históricos necesarios, organiza el análisis mediante varios agentes LLM y mantiene los cálculos dentro de un flujo programático controlado. El sistema separa la fase de datos, la planificación analítica, la generación y validación de código, la ejecución y la interpretación final, y conserva artefactos que permiten reconstruir cada ejecución completa.

1.2. Motivación

El proyecto nace del interés por combinar tres ámbitos que a menudo se estudian por separado: los modelos de lenguaje, la programación para el análisis de datos y la interpretación financiera. Los LLM permiten construir con rapidez interfaces capaces de entender peticiones diversas, pero esa misma facilidad puede ocultar errores cuando todo el proceso queda reducido a una única respuesta generativa.

La motivación principal es comprobar hasta qué punto puede aprovecharse la flexibilidad del modelo sin renunciar a mecanismos claros de control. En el sistema propuesto, el LLM no actúa como una caja negra que responde directamente, sino como parte de un flujo en el que primero se resuelven los datos, después se planifica el análisis, más tarde se genera y valida el código, y solo al final se redacta la respuesta. Esta separación permite revisar tanto el planteamiento seguido como los cálculos realmente obtenidos.

Otro criterio importante es que los fallos sean visibles. Si la petición de datos no puede resolverse con fiabilidad, si la descarga no devuelve información útil, si el código generado no es aceptable o si la ejecución no produce una salida válida, el sistema registra el problema y devuelve un resultado controlado. Este comportamiento resulta preferible a completar una respuesta con cifras no verificadas. La evaluación mediante una batería de consultas de distinta dificultad permite observar estos fallos e identificar en qué parte del proceso se producen.

El prototipo se construyó de forma incremental, desde consultas sencillas hasta una arquitectura multiagente con validaciones intermedias, reparación acotada y trazabilidad completa. La idea de fondo de este trabajo es que, en un dominio sensible como el financiero, la utilidad de un sistema basado en LLM depende no solo de que responda, sino de que pueda explicarse cómo se ha llegado a esa respuesta.

1.3. Objetivos

El objetivo general del trabajo es diseñar, implementar y evaluar un sistema basado en un modelo de lenguaje capaz de transformar consultas sobre datos financieros históricos en cálculos ejecutables y respuestas trazables.

Los objetivos específicos son:

- Diseñar una arquitectura modular que separe la resolución de datos, la planificación del análisis, la generación y validación de código, la ejecución de los cálculos y la elaboración de la respuesta final.
- Desarrollar un flujo capaz de interpretar consultas con diferentes requisitos analíticos y determinar de forma explícita qué datos, métricas y operaciones necesita cada caso.

- Resolver, validar y descargar datos históricos de mercado de forma controlada antes de pasar a la fase analítica.
- Generar y ejecutar código Python adaptado a cada consulta, aplicando controles que limiten errores de estructura, de ejecución y de formato de salida.
- Garantizar la trazabilidad del proceso mediante la conservación de contratos intermedios, código generado, resultados estructurados, artefactos de ejecución y registros de error.
- Generar respuestas financieras basadas exclusivamente en los cálculos realizados, sin incorporar predicciones ni recomendaciones de inversión.
- Evaluar la capacidad del sistema para comprender la consulta, seleccionar métricas pertinentes, ejecutar el análisis de forma correcta y producir respuestas claras, prudentes y fundamentadas.

1.4. Alcance

El sistema se limita al análisis histórico y descriptivo de activos financieros. Trabaja con datos de mercado obtenidos dentro del propio flujo y reutiliza también artefactos persistidos para pruebas, trazas y evaluación. Los casos utilizados incluyen acciones, índices, fondos cotizados, divisas, una materia prima y un criptoactivo, con diferentes periodos e intervalos temporales.

El prototipo no obtiene precios en tiempo real durante la ejecución, no incorpora noticias ni información macroeconómica y tampoco realiza análisis fundamental. No pretende predecir movimientos futuros ni recomendar la compra o venta de activos. Estas restricciones responden al propósito del trabajo: evaluar una arquitectura LLM trazable, no construir un asesor financiero.

1.5. Estructura de la memoria

La memoria se organiza en torno a tres ejes principales: la metodología, el flujo multiagente y la evaluación del sistema. El Capítulo 2 revisa los trabajos previos y sitúa la propuesta dentro del estado de la cuestión. El Capítulo 3 resume el material de trabajo y el planteamiento experimental de partida.

El Capítulo 4 define la metodología con la que se estudiará el sistema: unidad de análisis, protocolo de ejecución, criterios de evaluación y amenazas a la validez. El Capítulo 5 se reserva para el flujo multiagente del sistema y para la explicación detallada de sus etapas, contratos, validaciones y rutas de recuperación. El Capítulo 6 recogerá los resultados y la evaluación final del proyecto.

Los últimos capítulos se plantean de forma más compacta. El Capítulo 7 reunirá las limitaciones, los aspectos éticos y las condiciones de reproducibilidad, mientras que el Capítulo 8 cerrará la memoria con las conclusiones y el trabajo futuro. Por último, los anexos conservarán solo la información operativa de apoyo que siga siendo útil para ejecutar el sistema, documentar su configuración LLM y describir los artefactos técnicos que deja cada ejecución.

Capítulo 2

Estado de la cuestión

2.1. Enfoque de la revisión

Este capítulo sitúa el trabajo dentro del uso de modelos de lenguaje para el análisis financiero y la automatización de tareas de datos. No se pretende revisar todo el campo de la inteligencia artificial aplicada a las finanzas, sino centrarse en las líneas que sustentan el sistema desarrollado: la especialización de modelos en el dominio financiero, las arquitecturas basadas en agentes, el uso de herramientas externas, la generación de código como artefacto intermedio y la evaluación de sistemas generativos [13, 15].

El punto de partida del proyecto es un modelo abierto de propósito general, `openai/gpt-oss-20b`, integrado en un flujo de trabajo controlado [11]. Ante una consulta en lenguaje natural, el modelo participa en la planificación del análisis, la generación de código Python y la redacción de una respuesta apoyada en datos históricos locales. Quedan fuera del alcance el entrenamiento de un modelo propio, la predicción de precios y la toma autónoma de decisiones de inversión.

2.2. Modelos de lenguaje y agentes financieros

La aplicación del procesamiento del lenguaje natural al ámbito financiero es anterior a los modelos generativos actuales. FinBERT adapta BERT al lenguaje financiero y muestra la utilidad de los modelos especializados en tareas como el análisis de sentimiento [1]. Posteriormente, BloombergGPT y FinGPT ampliaron esta línea mediante modelos entrenados o ajustados con noticias, documentación corporativa y otros recursos propios del sector [16, 19].

Estos trabajos muestran que el lenguaje financiero incorpora terminología, relaciones y matices que conviene tratar con cuidado. Sin embargo, también sugieren que la especialización del modelo no es la única vía posible. En este trabajo se adopta una estrategia complementaria: se emplea un modelo generalista de pesos abiertos dentro

de una arquitectura que restringe entradas, valida artefactos y ancla la respuesta en datos observables [11]. El interés, por tanto, no está en comparar modelos aislados, sino en estudiar el valor de su integración en un sistema verificable de análisis histórico.

La evolución reciente de los LLM financieros también ha favorecido la aparición de sistemas basados en agentes. FinRobot propone una arquitectura orientada a la investigación y valoración de empresas, mientras que TradingAgents distribuye tareas de análisis, debate y toma de decisiones de *trading* entre varios agentes [17, 21]. En ambos casos, el modelo deja de actuar solo como generador de texto y pasa a intervenir en un proceso compuesto por distintas responsabilidades. El proyecto comparte esa idea de separación funcional, aunque con un objetivo más acotado: analizar datos históricos de forma trazable, sin operar sobre el mercado ni recomendar inversiones.

2.3. Agentes, herramientas y orquestación

Los sistemas basados en agentes permiten que un modelo vaya más allá de la generación de texto: puede utilizar herramientas y trabajar con los resultados de acciones externas. ReAct propone alternar razonamiento y actuación para que el modelo observe las consecuencias de sus decisiones [20]. Toolformer estudia cómo un modelo puede aprender a recurrir a herramientas cuando las necesita [14]. Las revisiones sobre agentes basados en LLM describen esta evolución como el paso de sistemas puramente conversacionales a arquitecturas capaces de planificar, actuar y aprovechar observaciones del entorno [15].

Para coordinar estas tareas es necesario conservar la información producida durante la ejecución. En la práctica, esto exige mantener un estado compartido, definir etapas con responsabilidades separadas y establecer transiciones controladas entre validación, generación, ejecución e interpretación. Esta estructura resulta especialmente útil cuando se combinan componentes generativos con comprobaciones programáticas, porque permite distinguir con más claridad qué parte del proceso ha fallado y qué evidencias quedan disponibles para revisarlo.

El sistema desarrollado adopta ese patrón, pero de forma deliberadamente acotada. El modelo no selecciona libremente qué herramienta usar ni altera el orden global del proceso. En su lugar, opera dentro de un recorrido fijo con estado compartido y transiciones explícitas entre resolución de datos, planificación, generación de código, validación previa a la ejecución, ejecución controlada e interpretación final. Esta decisión reduce la libertad operativa, pero refuerza el control y la trazabilidad, que son dos prioridades centrales del trabajo.

2.4. Generación y ejecución controlada de código

La generación automática de código es una de las capacidades que más ha impulsado el uso de los modelos de lenguaje en tareas técnicas. Uno de los trabajos de referencia en esta línea fue Codex, que ayudó a consolidar la idea de que un modelo de lenguaje puede recibir una instrucción escrita en lenguaje natural, transformarla en un programa y comprobar después si ese programa funciona mediante pruebas funcionales [3]. En ciencia de datos, DS-1000 amplió esta línea con un banco de pruebas más cercano a escenarios reales y apoyado en librerías habituales de análisis [7].

Una idea especialmente relevante para este proyecto es la de PAL, donde el modelo no resuelve todo el problema en lenguaje natural, sino que transforma la petición en un programa ejecutable y delega el cálculo efectivo en un intérprete [5]. Esta perspectiva encaja de forma directa con la arquitectura desarrollada: aquí el código no se trata como una respuesta final, sino como un artefacto intermedio que debe ser validado, ejecutado y contrastado antes de redactar la respuesta al usuario.

La literatura reciente también ha mostrado que la reparación iterativa puede mejorar los resultados cuando el primer intento falla. Self-Debugging explora cómo un modelo puede revisar su propio programa a partir de su explicación y de la evidencia de ejecución [4]. Sin reproducir exactamente ese planteamiento, el sistema retoma la misma intuición: cuando aparecen errores reparables, conviene devolver al modelo un contexto acotado del fallo y exigir una nueva versión del script bajo restricciones explícitas.

Ejecutar código generado por un modelo introduce, además, riesgos específicos. Los estudios sobre asistentes de programación han mostrado que sus sugerencias pueden contener vulnerabilidades, incluso cuando parecen funcionales [12]. Por este motivo, el código no se acepta por mera plausibilidad. La propuesta lo obliga a respetar un contrato estructurado, lo somete a una validación previa y limita el espacio de operaciones permitidas antes de su ejecución efectiva.

2.5. Fiabilidad y evaluación de sistemas generativos

Una limitación conocida de los modelos generativos es la producción de respuestas plausibles que no se corresponden con la información disponible, fenómeno denominado habitualmente alucinación [6]. La generación aumentada con recuperación busca reducir este problema apoyando la respuesta en fuentes obtenidas de forma explícita [8]. Aunque el sistema no incorpora recuperación documental, comparte el mismo principio de anclaje: la explicación final debe apoyarse en datos locales, código ejecutado y resultados registrados.

Este anclaje es especialmente importante en el análisis financiero. Cuando una res-

puesta presenta datos de rentabilidad, volatilidad o *drawdown*, debe ser posible revisar tanto la información de partida como el procedimiento seguido. También conviene separar con claridad la descripción del comportamiento histórico de cualquier predicción o recomendación. Conservar la consulta, el plan, el código, los resultados, los errores y la respuesta final no elimina la posibilidad de fallo, pero ayuda a localizarlo y a valorar sus consecuencias.

La evaluación de un sistema de estas características no puede reducirse a una única cifra. HELM defiende marcos de evaluación amplios y transparentes, apoyados en varios escenarios y métricas, para evitar que un resultado agregado oculte limitaciones relevantes [9]. De forma complementaria, la revisión de evaluación útil de LLM distingue entre valorar la capacidad nuclear del modelo y valorar su comportamiento como agente integrado en un sistema [13]. Esta distinción resulta muy pertinente en este trabajo, porque aquí importan tanto la calidad de la salida final como la robustez del flujo, la pertinencia del plan, la corrección del código y la trazabilidad de cada ejecución.

En el caso del texto generado, también se ha señalado que las métricas automáticas tradicionales no siempre reflejan bien la calidad percibida durante una revisión humana [10]. Siguiendo estas ideas, la evaluación del proyecto combina indicadores técnicos, revisión cualitativa y lectura humana de resultados. Esta aproximación permite distinguir entre los fallos del flujo y las respuestas que, aun completándose, presentan problemas de pertinencia, fidelidad o claridad.

2.6. Comparativa y posicionamiento de la propuesta

La Tabla 2.1 resume las líneas revisadas y muestra cómo se relacionan con el sistema desarrollado. El objetivo concreto es integrar un modelo de lenguaje con generación controlada de código y análisis reproducible de datos históricos.

Tabla 2.1: Comparación entre los trabajos relacionados y la propuesta desarrollada.

Trabajo o línea	Aportación principal	Relación con este trabajo
FinBERT, BloombergGPT y FinGPT [1, 16, 19]	Modelos entrenados o adaptados al lenguaje financiero.	Se centran en el modelo; la propuesta estudia la integración controlada de un modelo abierto generalista dentro de un flujo verificable.
FinRobot y TradingAgents [17, 21]	Agentes para investigación, valoración o <i>trading</i> .	Inspiran la separación de responsabilidades, pero el sistema no toma decisiones de inversión ni actúa sobre el mercado.
ReAct, Toolformer y revisiones de agentes [14, 15, 20]	Uso de herramientas externas y coordinación de pasos mediante observaciones y transiciones controladas.	Respaldan el uso de herramientas y un estado común; aquí el recorrido está fijado para favorecer el control.

Trabajo o línea	Aportación principal	Relación con este trabajo
Codex, DS-1000, PAL y Self-Debugging [3–5, 7]	Síntesis de código, evaluación en escenarios técnicos y reparación iterativa de programas.	Justifican tratar el código como artefacto intermedio, validarlo antes de ejecutarlo y permitir reparaciones acotadas cuando aparecen errores.
Seguridad de código generado [12]	Riesgos de vulnerabilidades o implementaciones inseguras en sugerencias producidas por modelos.	Refuerza la necesidad de restringir operaciones, validar el script y no ejecutar código por mera plausibilidad.
Alucinaciones, RAG y evaluación de LLM [6, 8–10, 13]	Estudio de respuestas no fieles, mecanismos de anclaje y evaluación multidimensional del comportamiento del sistema.	Fundamentan la combinación de anclaje a datos locales, indicadores técnicos, dimensiones cualitativas y revisión humana.

Capítulo 3

Material de trabajo y planteamiento experimental

Este capítulo sitúa el punto de partida práctico del trabajo antes de entrar en la metodología formal y antes de describir con detalle el flujo multiagente. Su función es precisar con qué material se construye el sistema, qué clase de consultas se quiere estudiar y por qué ese conjunto resulta suficiente para poner a prueba la arquitectura en condiciones realistas.

3.1. Origen y naturaleza del material de trabajo

La fuente principal de datos históricos utilizada en el proyecto es Yahoo Finance, consumida a través de la librería `yfinance` [2, 18]. Esta elección permite trabajar con una base amplia de instrumentos y facilita que el sistema resuelva consultas abiertas sin depender de un pequeño catálogo manual preparado de antemano. La arquitectura se plantea como un flujo capaz de interpretar la consulta del usuario, convertirla en una petición estructurada y comprobar después si los datos históricos necesarios pueden descargarse de forma operativa.

Esto no significa que el sistema pueda analizar cualquier cosa sin restricciones. El alcance real depende de que el instrumento solicitado pueda identificarse con suficiente claridad, de que exista histórico accesible en `yfinance` y de que la petición sea compatible con un análisis basado en precios y volumen. Dicho de otro modo, el trabajo no fija una lista cerrada de activos, pero sí se mueve dentro del universo de consultas que pueden resolverse de manera razonable con datos históricos de mercado.

Para estudiar ese comportamiento se ha buscado una combinación de apertura y representatividad. Por un lado, la entrada del sistema sigue siendo una consulta libre en lenguaje natural. Por otro, el proyecto se apoya en familias de casos que permiten tensionar la arquitectura de forma controlada. En la práctica, esto incluye

ejemplos sobre acciones, fondos cotizados, índices, divisas, una materia prima y un criptoactivo, con distintos horizontes temporales y con granularidad diaria e intradía. Esta variedad no pretende cubrir todo el mercado financiero, pero sí ofrecer un conjunto lo bastante heterogéneo como para observar cómo responde el sistema ante consultas simples, comparativas y más exigentes.

Otro rasgo importante es que los datos no se consideran solo una entrada pasiva, sino una parte activa del experimento. La fase de datos debe resolver el instrumento, validar la petición y completar una descarga real antes de que comience la parte analítica. Por eso el material de trabajo no se reduce a la consulta del usuario ni a la respuesta final: incluye también los artefactos generados durante la descarga, la normalización, la ejecución del código y la traza completa del proceso.

3.2. Casos y escenario experimental de partida

Durante la construcción del sistema se utilizaron varios ejemplos representativos para comprobar si cada fase iba funcionando como se esperaba. Estos ejemplos no constituyen la evaluación final, pero sí ayudan a entender el tipo de material empleado para poner a prueba el proyecto. Entre ellos aparecen consultas de crecimiento simple sobre un único activo, comparaciones entre dos instrumentos, lecturas generales de corto plazo, análisis de retorno y riesgo, y algunos casos con mayor carga interpretativa o con restricciones de formato más explícitas. En este sentido, los ejemplos de desarrollo sirvieron como banco de pruebas incremental para verificar que la fase de datos, la planificación analítica, la generación de código, la validación y la interpretación final mantenían una coherencia razonable.

La evaluación principal se articula, sin embargo, alrededor de una batería de 50 consultas organizada en tres niveles de complejidad. El nivel A agrupa consultas directas, centradas en una o dos métricas fáciles de identificar y con un formato de respuesta simple. El nivel B introduce comparaciones más ricas, varias métricas relevantes o una estructura de salida algo más elaborada. El nivel C reúne los casos más exigentes: informes más completos, restricciones explícitas de presentación o peticiones donde el sistema debe mantener mejor la disciplina de salida y la prudencia interpretativa.

La distribución prevista de la batería no es equilibrada entre niveles, sino intencionadamente más exigente: 10 casos de nivel A, 20 de nivel B y 20 de nivel C. La razón es sencilla. Aunque parece razonable esperar mejor rendimiento en las consultas más fáciles, interesa comprobar qué ocurre cuando aumenta la dificultad y la respuesta exige una mejor coordinación entre la fase de datos, la parte analítica y la interpretación final. La batería no se diseña solo para confirmar que el sistema responde en casos sencillos, sino para identificar en qué punto empieza a degradarse su comportamiento y qué errores emergen cuando el escenario se vuelve más complejo.

Aunque la entrada del sistema siga siendo abierta, las consultas que se quieren estudiar pueden agruparse en varias familias. La primera corresponde a consultas descriptivas directas, como crecimiento acumulado, rango de precios o resumen general de un activo en un periodo dado. La segunda incluye comparativas entre instrumentos, por ejemplo retorno frente a riesgo, *drawdown*, correlación o liderazgo relativo. La tercera se apoya en lecturas algo más técnicas, como medias móviles, perfil de volatilidad o comportamiento por subperiodos. La cuarta incorpora restricciones explícitas de formato o de estilo, por ejemplo separar métricas y limitaciones, redactar para un usuario no técnico o devolver estructuras concretas de salida. Por último, algunos casos fuerzan al sistema a reconocer mejor los límites del propio análisis histórico, evitando noticias, predicciones o recomendaciones de inversión.

En conjunto, este planteamiento experimental resulta suficiente para justificar el estudio por tres razones. En primer lugar, el material utilizado refleja el comportamiento real del sistema, que depende de consultas abiertas y de descargas efectivas de datos históricos. En segundo lugar, la variedad de activos y de tareas permite observar el flujo en situaciones distintas sin convertir el trabajo en una exploración inabarcable. En tercer lugar, la batería escalonada por niveles de dificultad no solo mide si el sistema acierta, sino también cómo cambia su rendimiento cuando la consulta exige más cálculo, más estructura y una interpretación más cuidadosa.

Capítulo 4

Metodologia

4.1. Funcion de este capitulo

Este capitulo debe explicar como se va a estudiar el sistema y con que criterio se va a juzgar si el trabajo funciona bien o no. No debe confundirse con la descripcion del flujo: aqui lo importante no es como esta construido el sistema, sino como se va a analizar de forma defendible.

4.2. Enfoque metodologico

Completar aqui la idea central del enfoque metodologico. Por ejemplo: desarrollo iterativo de un sistema multiagente, evaluacion empirica sobre casos de analisis financiero historico, combinacion de revision tecnica y lectura cualitativa de resultados, etc.

4.3. Preguntas guia para escribir bien la metodologia

4.3.1. Sobre el objetivo de evaluacion

1. ¿Que quieres demostrar exactamente con este trabajo?
2. ¿Quieres demostrar viabilidad del flujo, robustez de la arquitectura, calidad de las respuestas o una combinacion de ellas?
3. ¿Que seria un buen resultado, pero no necesariamente una demostracion completa?
4. ¿Que afirmaciones no deberias hacer aunque el prototipo funcione bien?

4.3.2. Sobre la unidad de analisis

1. ¿Que vas a considerar como unidad de analisis principal: una consulta, una ejecucion completa, una etapa del flujo o un caso compuesto?
2. ¿Te interesa medir solo el resultado final o tambien el comportamiento intermedio de cada agente?
3. ¿Quieres analizar los fallos por etapa o solo por caso completo?

4.3.3. Sobre el diseno experimental

1. ¿Como vas a construir o seleccionar la bateria de consultas del trabajo?
2. ¿Como vas a graduar la dificultad de los casos?
3. ¿Quieres hablar de complejidad de forma cualitativa sin imponer una etiqueta fija al sistema?
4. ¿Que tipos de consulta deben estar presentes para que el experimento sea creible?
5. ¿Que parte del flujo quieres tensionar de forma deliberada?

4.3.4. Sobre la evaluacion

1. ¿Como distinguiras entre exito tecnico del flujo y calidad real del analisis?
2. ¿Que metricas tecnicas quieres observar: completitud, tiempo, errores por etapa, reparaciones, trazas conservadas?
3. ¿Que dimensiones cualitativas quieres valorar: pertinencia de metricas, fidelidad de la interpretacion, prudencia, claridad?
4. ¿Habra revision humana? Si la hay, ¿con que rubrica o con que criterios?

4.3.5. Sobre la validez del estudio

1. ¿Que amenazas a la validez interna puede tener el experimento?
2. ¿Que limita la generalizacion del trabajo?
3. ¿Hasta que punto dependes del modelo concreto, de los prompts o del proveedor de inferencia?
4. ¿Como evitaras confundir una respuesta convincente con un resultado correcto?

4.4. Estructura recomendada para la metodologia final

4.4.1. Pregunta de investigacion y objetivos evaluables

Completar aqui la pregunta principal de investigacion y los objetivos que realmente puedan verificarse con el experimento.

4.4.2. Unidad de analisis

Completar aqui que se evalua exactamente en cada caso y por que esa unidad de analisis es adecuada para el trabajo.

4.4.3. Diseno experimental

Completar aqui el numero de casos, su clasificacion, su dificultad y el razonamiento que justifica ese diseno. Si se habla de consultas mas simples o mas avanzadas, conviene que esa clasificacion se use solo como herramienta de evaluacion y no como una etiqueta explicita que el sistema reciba en la entrada.

4.4.4. Protocolo de ejecucion

Completar aqui como se lanza cada caso, que artefactos se conservan, que pasos del flujo deben completarse y como se registran errores, reparaciones y tiempos.

4.4.5. Metricas de evaluacion

Metrica	Tipo	Utilidad dentro de la evaluacion
Completar	Tecnica / cualitativa	Completar
Completar	Tecnica / cualitativa	Completar
Completar	Tecnica / cualitativa	Completar

Tabla 4.1: Metricas previstas para la evaluacion del sistema.

4.4.6. Revision humana

Completar aqui si habra rubricado manual, lectura comparada de respuestas, comprobacion de metricas o supervision por etapas.

4.4.7. Amenazas a la validez

Completar aqui sesgos de seleccion de casos, dependencia de configuracion, sensibilidad a prompts y cualquier otra limitacion metodologica importante.

4.5. Criterio de calidad para este capitulo

La metodologia final deberia permitir responder sin ambigüedad a cuatro cuestiones:

1. ¿Que se va a evaluar exactamente?
2. ¿Como se va a ejecutar esa evaluacion?
3. ¿Con que criterios se juzgara el sistema?
4. ¿Que limites tendra la interpretacion de los resultados?

Capítulo 5

Flujo multiagente del sistema

5.1. Funcion de este capitulo

Este capitulo es uno de los nucleos del trabajo. Aqui no basta con decir que el sistema usa varios agentes: hace falta explicar con detalle como circula la informacion, que hace cada bloque, que controles existen y por que esa organizacion mejora la trazabilidad y la explicabilidad del analisis.

La idea central que conviene sostener en todo el capitulo es la siguiente: el valor del sistema no esta solo en producir una respuesta final, sino en hacerlo mediante un flujo auditable, con artefactos observables y con decisiones tecnicas que permitan entender que ha ocurrido en cada etapa.

5.2. Preguntas que debe responder el capitulo

1. ¿Que recibe exactamente el sistema y como se transforma esa entrada a lo largo del flujo?
2. ¿Que hace cada bloque principal y por que ese bloque es necesario?
3. ¿Que diferencia hay entre el flujo principal y los mecanismos de correccion?
4. ¿Que papel tienen los prompts como contrato operativo de los agentes?
5. ¿Que evidencias deja el sistema para poder reconstruir una ejecucion completa?
6. ¿Por que este flujo es mas explicable y controlable que una respuesta directa del LLM?
7. ¿Que decisiones de diseno ayudan a justificar luego la metodologia y la evaluacion?

5.3. Vision general del flujo

La Figura 5.1 se toma como referencia de alto nivel para describir el sistema. En ella se representa el recorrido completo desde la solicitud inicial del usuario hasta la respuesta final, incluyendo los agentes principales, los subagentes de correccion y los puntos de validacion mas relevantes.

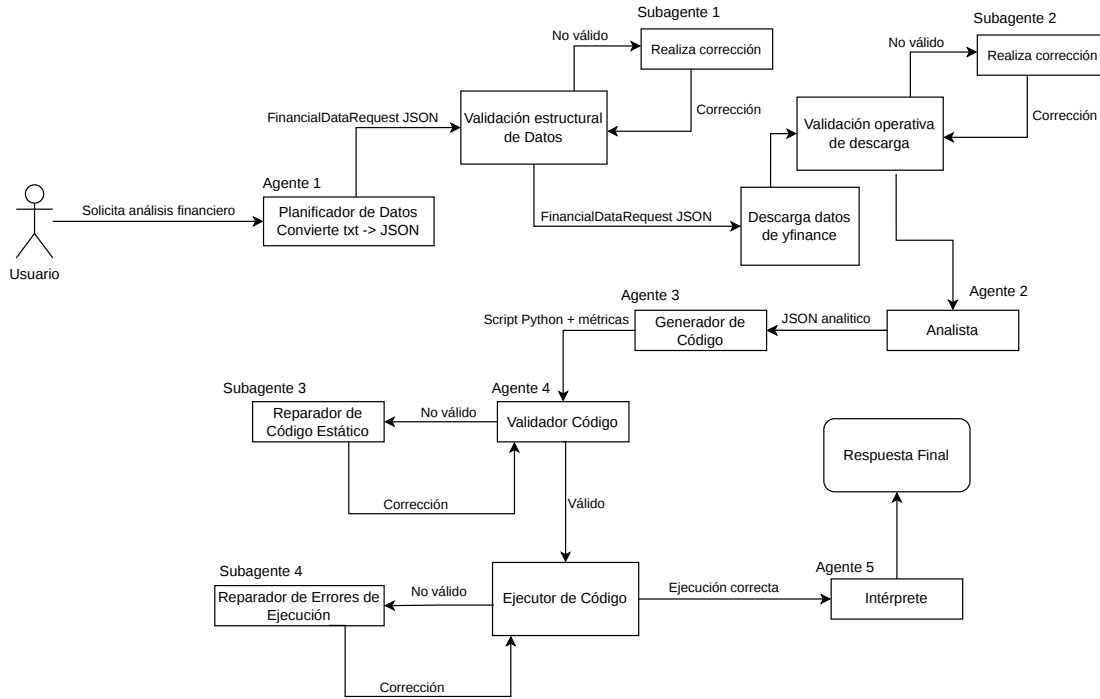


Figura 5.1: Flujo multiagente de referencia para el desarrollo del sistema de analisis financiero.

El flujo parte de una consulta escrita en lenguaje natural y la transforma de manera progresiva en una respuesta final sustentada por datos descargados, decisiones intermedias trazables y una ejecución controlada. La primera fase se concentra en resolver el problema de datos: el Agente 1 convierte la petición del usuario en un `FinancialDataRequest` estructurado, este contrato se valida a nivel formal y, después, se comprueba operativamente mediante una descarga real en `yfinance`. Solo cuando ambas comprobaciones se superan se considera que el sistema dispone de una base suficientemente fiable para pasar al análisis.

La fase central del flujo comienza entonces con el Agente 2, que recibe los datos ya descargados y los convierte en un JSON analítico que acota que cálculos deben realizarse. A partir de ese plan, el Agente 3 genera el script Python encargado de materializar el análisis. Ese script no se ejecuta directamente: primero pasa por el Agente 4, que valida su estructura y sus restricciones, y solo después se lanza en el

ejecutor. Finalmente, el Agente 5 interpreta la salida correcta y la convierte en una respuesta legible para el usuario.

La arquitectura incorpora además cuatro puntos de corrección, uno por cada familia de fallos relevantes: errores estructurales en la petición de datos, errores operativos de descarga, errores estáticos del código generado y errores observados durante la ejecución. Esta separación de responsabilidades no solo ordena el sistema, sino que mejora su explicabilidad. Cada bloque tiene una entrada definida, una salida esperada y un criterio claro de éxito o bloqueo, lo que facilita defender por qué el flujo avanza, por qué se repara o por qué se detiene en un caso concreto.

5.4. Entrada y estado compartido

5.4.1. Entrada del sistema

El sistema arranca a partir de una estructura de entrada ligera, representada internamente como `FinancialQueryInput`. El único campo estrictamente obligatorio en ese punto es la consulta original del usuario, es decir, la petición en lenguaje natural sobre la que debe trabajar el flujo. Junto a ella pueden aparecer, si ya se conocen, algunos elementos de contexto como tickers sugeridos, un rango temporal, un periodo, un intervalo o rutas de CSV previamente disponibles. Sin embargo, estos campos auxiliares no sustituyen a la consulta principal ni se consideran conocimiento plenamente validado hasta que la fase de datos los revisa.

Esta decisión de diseño es importante porque evita que el sistema parta de supuestos opacos. Ni el Agente 1 ni los agentes posteriores deben asumir libremente que un ticker es correcto, que un periodo está bien resuelto o que existe ya un fichero útil si esa información no ha quedado consolidada dentro del flujo. La consulta del usuario es la fuente primaria de intención; el resto del contexto solo se convierte en contrato operativo después de la validación correspondiente.

También conviene subrayar que la entrada no incluye una etiqueta externa que indique de antemano la profundidad analítica esperada. El sistema no recibe una clasificación previa del tipo “consulta simple” o “consulta avanzada”. Esa graduación debe inferirse a partir del contenido de la petición, de sus restricciones y del formato de salida que se desprenda de ella. Este detalle es metodológicamente relevante porque muestra que el comportamiento de los agentes se apoya en el significado de la consulta y no en una señal manual añadida fuera del flujo.

Desde el punto de vista de la calidad, una entrada bien delimitada reduce varios riesgos tempranos: ambigüedades sobre el instrumento, incoherencias temporales, peticiones imposibles de descargar y errores derivados de trasladar al análisis hipótesis que aun no han sido comprobadas. Por eso la memoria puede defender que el sistema

no empieza “analizando”, sino primero asegurando que la petición de partida tiene una forma utilizable y auditable.

5.4.2. Estado compartido

La ejecución se articula alrededor de un estado compartido tipado, `WorkflowState`, que actúa como memoria operativa del flujo. Su función no es solo transportar datos entre nodos, sino conservar artefactos y decisiones intermedias para que la respuesta final pueda reconstruirse después. Gracias a este estado común, cada fase trabaja sobre resultados ya consolidados de la fase anterior y deja, a su vez, una salida observable para las siguientes.

Un primer bloque del estado recoge la intención original y su normalización progresiva. Aquí se guardan la consulta del usuario y un contexto resuelto que va acumulando `tickers`, fechas, periodo, intervalo y rutas de CSV conforme la fase de datos los valida. Este grupo de campos permite distinguir entre lo que el usuario pidió inicialmente y lo que el sistema ha conseguido concretar de forma operativa.

Un segundo bloque conserva los artefactos de la fase de datos: el `FinancialDataRequest` validado, el resumen de descarga y las rutas de los ficheros persistidos. Esta información es la que hace posible enlazar la fase 1 con la fase 2 de manera explícita. El analista y el generador de código no reciben una consulta vaga, sino un contexto derivado de estos artefactos, resumido en un payload compacto pero respaldado por el estado completo.

El tercer bloque corresponde al núcleo de esta parte del flujo: `analysis_plan` y `generated_code`. El primero recoge el contrato analítico decidido por el Agente 2; el segundo almacena el script Python producido por el Agente 3. Ambos campos son esenciales para la trazabilidad interna, porque permiten separar que se quiso calcular de como se implementó técnicamente.

Por último, el estado incorpora la salida de ejecución, la respuesta final y los mecanismos de control. Aquí se incluyen `execution_stdout`, `execution_stderr`, `execution_output`, los artefactos de ejecución, la respuesta final en lenguaje natural, la lista de avisos acumulados, el mensaje de error si lo hubiera y el estado textual del workflow. Este conjunto permite reconstruir no solo los casos exitosos, sino también en qué punto se bloqueó el flujo y con qué motivo.

En consecuencia, el valor del estado compartido no reside únicamente en “pasar variables” entre nodos. Su papel principal es convertir una secuencia de llamadas a agentes y validadores en un proceso auditable, donde cada transición deja una evidencia concreta y donde la respuesta final puede justificarse a partir de artefactos observables.

5.5. Flujo principal del sistema

Este bloque debe contar el recorrido principal del sistema de forma limpia, sin detenerse aun en todos los bucles de correccion. La idea es explicar que hace cada fase, que recibe, que produce y por que es necesaria dentro del flujo completo.

5.5.1. Parte de datos

La parte de datos cubre el tramo que va desde la consulta inicial del usuario hasta la obtencion de una descarga valida lista para el Agente 2. Su funcion no es producir aun el analisis financiero, sino delimitar con precision sobre que informacion va a poder trabajar el sistema. Esta distincion es importante: antes de hablar de metricas, comparaciones o interpretaciones, el flujo debe demostrar que ha entendido que datos necesita, que puede pedirlos de forma coherente y que la fuente externa responde con una descarga util.

El primer paso de esta fase corresponde al Agente 1, que actua como planificador de datos. A partir de la consulta en lenguaje natural, este agente genera un `FinancialDataRequest` en formato JSON. Ese contrato debe recoger, al menos, los instrumentos o tickers detectados, la granularidad temporal, el rango o periodo solicitado y la indicacion de si la peticion esta lo bastante clara como para continuar. El objetivo aqui no es inferir conclusiones financieras, sino traducir la intencion del usuario a una peticion de datos concreta, utilizable por el resto del sistema y suficientemente explicita como para ser revisada.

Una vez generado ese request, el flujo no pasa de inmediato a la descarga. Primero se realiza una validacion estructural. En esta comprobacion se revisa que el JSON exista realmente, que sus campos obligatorios esten presentes, que los tipos sean correctos y que no haya incoherencias obvias entre fechas, periodos, intervalos o lista de instrumentos. Tambien se comprueba que la peticion no este apoyada en supuestos demasiado debiles. Si el request contiene errores corregibles, se activa el Subagente 1, cuya tarea consiste en reescribirlo de forma controlada y devolver una nueva version para validacion. Si el problema no es corregible con fiabilidad, o si se agotan los intentos permitidos, el flujo se bloquea y no avanza a etapas posteriores.

Superada la validacion estructural, la fase de datos entra en su segunda comprobacion: la validacion operativa de descarga. Aqui se intenta la descarga real en `yfinance` con los parametros ya validados. Esta decision es metodologicamente relevante, porque no basta con que la peticion sea correcta en teoria. Tambien debe demostrarse que produce un resultado util en la practica. En esta etapa se verifica que la llamada no devuelve un conjunto vacio, que los instrumentos esperados aparecen efectivamente y que la respuesta permite persistir artefactos de datos reutilizables por el sistema. Si esta descarga falla de forma reparable, interviene el Subagente 2, especializado en ajustar

la peticion desde un punto de vista operativo; si no existe una correccion razonable, la ejecucion vuelve a detenerse.

El resultado final de esta fase es, por tanto, mas fuerte que un simple “request valido”. Lo que se entrega al Agente 2 es una descarga ya realizada, respaldada por un **FinancialDataRequest** previamente comprobado y acompanada de los artefactos necesarios para reconstruir el proceso. Desde el punto de vista de la explicabilidad, esta organizacion permite defender que el sistema no empieza a analizar ni a generar codigo sobre una interpretacion vaga de la consulta, sino sobre datos cuya necesidad, forma de obtencion y viabilidad han quedado justificadas antes de continuar.

5.5.2. Analista y generador de codigo

Una vez superada la fase de datos, el flujo entra en un segundo bloque conceptual: convertir una peticion ya respaldada por datos descargados en un procedimiento analitico ejecutable. Esta transformacion se divide en dos agentes consecutivos para separar con claridad dos responsabilidades distintas. El Agente 2 decide que debe calcularse; el Agente 3 decide como materializarlo en Python. Esta division no es ornamental, sino una medida de control. Evita pedir al mismo bloque que razone el analisis, improvise la implementacion y produzca a la vez una conclusion final.

El Agente 2 actua como analista. Su entrada no es la consulta bruta, sino un traspaso compacto construido a partir del estado de la fase 1. Ese payload contiene la consulta original, los tickers resueltos, el contexto temporal, las rutas de los CSV normalizados y un resumen minimo de los datos reales disponibles, incluyendo numero de filas y columnas observadas. Sobre esa base, el agente genera un **AnalysisPlan** estructurado. Este plan fija el objetivo analitico, el tipo de analisis, las metricas que deben calcularse, las columnas requeridas, las restricciones de datos, las exigencias de salida y una justificacion breve de por que ese contrato responde a la consulta. El resultado no es una respuesta financiera, sino una especificacion intermedia revisable.

La transicion entre el Agente 2 y el Agente 3 se mantiene deliberadamente directa. El plan se parsea, se tipa y se guarda en el estado compartido como contrato analitico de la siguiente fase, pero no se inserta entre ambos agentes una puerta local de validacion semantica separada del flujo visible. Esta decision busca que la arquitectura se mantenga fiel a la figura general del sistema: el Agente 2 planifica, el Agente 3 implementa y la aceptacion o bloqueo del codigo pertenece al bloque posterior de validacion, no a una compuerta intermedia oculta dentro de la parte analitica.

Con ese plan ya tipado interviene el Agente 3, encargado de generar el script Python. Su entrada combina tres piezas: la consulta original, el mismo payload compacto que vio el analista y el **AnalysisPlan** ya generado por el Agente 2. El generador no debe reinterpretar la consulta desde cero ni decidir nuevas metricas, sino implementar el contrato

recibido. Para ello se le impone un prompt con restricciones técnicas explícitas: el script debe leerse desde `argv[1]`, cargar el payload mediante `Path(...).read_text(...)`, usar los cargadores permitidos del proyecto para acceder a los CSV, evitar accesos libres como `open()` o `pd.read_csv()` y producir un único JSON por `stdout`. Además, el prompt delimita imports permitidos, claves esperadas en la salida y reglas de comportamiento cuando falten datos.

Esta relación entre plan y script es uno de los puntos más defendibles de la arquitectura. El **AnalysisPlan** actúa como contrato entre razonamiento e implementación: reduce la ambigüedad, facilita revisar si el código calcula lo que se pidió y permite aislar errores de planificación de errores de programación. La salida del Agente 3 se guarda primero en memoria compartida como **generated_code**; después, cuando la parte de ejecución toma el relevo, ese mismo contenido se persiste como fichero en `results/code/generated_<run_id>.py`. De esta manera, la fase no entrega una conclusión opaca, sino dos artefactos trazables: un plan analítico y un script verificable.

Los riesgos principales de esta traducción aparecen precisamente en la frontera entre ambos agentes: planes demasiado vagos, columnas pedidas que no existen realmente, scripts que ignoran el contrato analítico o implementaciones que incumplen las reglas de carga y salida. Por eso esta parte del flujo no termina en la generación del código, sino en la preparación de un artefacto que debe someterse a la validación de la parte 3 y, después, a la validación de ejecución. En términos metodológicos, esto permite defender que el sistema no pide al LLM una recomendación financiera directa, sino una secuencia controlada de contratos y evidencias intermedias.

5.5.3. Validación del código

La siguiente etapa del flujo corresponde al Agente 4, encargado de validar el código generado antes de permitir su ejecución. Esta validación se formula también como una tarea LLM con salida estructurada. El agente no genera un script nuevo ni ejecuta el código para comprobarlo empíricamente. Su trabajo consiste en leer la consulta original, el contexto operativo heredado de la fase de datos, el **AnalysisPlan** producido por el Agente 2 y el script generado por el Agente 3, y decidir si ese código puede aceptarse, si necesita una corrección o si debe bloquearse.

La salida del Agente 4 se expresa mediante una decisión tipada con tres estados posibles: **valid**, **repairable** y **blocked**. Junto a esa etiqueta, el agente devuelve errores observados, avisos no bloqueantes, correcciones requeridas y un razonamiento breve. Esta forma de salida es importante porque evita respuestas vagas del tipo “el código parece bien” y obliga a dejar una justificación reutilizable por el resto del workflow. El estado compartido conserva además esta decisión como artefacto intermedio, lo que mejora la trazabilidad de la parte 3.

La validacion se centra en varias preguntas. En primer lugar, si el script implementa realmente el plan recibido y no otro problema distinto. En segundo lugar, si respeta el contexto de entrada ya resuelto y no introduce supuestos ocultos ajenos al payload del sistema. En tercer lugar, si mantiene un contrato de salida estructurada, de modo que la ejecucion posterior y el interprete final puedan trabajar sobre un JSON coherente con el workflow. Por ultimo, el agente debe valorar si los defectos observados siguen siendo corregibles o si la desviacion respecto al plan es demasiado grande como para seguir reintentando.

Esta etapa da lugar al Subagente 3 cuando el caso es **repairable**. El subagente no recibe una orden difusa de “arreglar el codigo”, sino el detalle de la decision anterior: errores, fixes requeridos y razonamiento del Agente 4. Sobre esa base devuelve un script completo nuevo, que vuelve a pasar por el Agente 4 para una segunda validacion. Si la nueva version se acepta, el flujo continua; si no converge tras el numero maximo de intentos o si el propio Agente 4 concluye que no existe una correccion razonable, el caso pasa a **blocked** y no llega al ejecutor.

Desde el punto de vista del diseno, este bloque permite sostener una idea importante del trabajo: generar codigo no es suficiente. Hace falta una etapa intermedia que valore si ese codigo encaja con el contrato del sistema antes de ejecutarlo. Al expresar esa valoracion como una decision estructurada de un agente especializado, la arquitectura mantiene la coherencia con el resto del flujo multiagente y evita esconder esta responsabilidad dentro de validadores locales opacos o compuertas no representadas en la figura principal.

5.5.4. Validacion de ejecucion

Una vez que el codigo ha sido aceptado por el Agente 4, el flujo entra en una fase distinta de las anteriores: ya no se trata de decidir si el script parece razonable en abstracto, sino de comprobar si realmente puede ejecutarse dentro del sistema y si la salida resultante sirve para las etapas posteriores. Esta distincion es importante porque un script puede haber superado la validacion estatica y, aun asi, fallar al cargar los datos, exceder el tiempo esperado, producir una excepcion en tiempo de ejecucion o devolver una salida que no encaje con el contrato del workflow.

La parte 4 comienza construyendo un payload operativo compacto a partir del estado compartido. Ese payload incluye la consulta original, los tickers resueltos, el contexto temporal, las rutas de los CSV normalizados y un resumen minimo de los datos disponibles. Su funcion no es volver a explicar que analisis habia que hacer, sino proporcionar al script el contexto practico necesario para ejecutarse sobre datos locales. En coherencia con esta responsabilidad, la fase de ejecucion no recibe **analysis_plan** dentro del payload lanzado al proceso: el contrato analitico ya se resolvió antes y la

pregunta que ahora interesa es puramente operativa, esto es, si el código corre bien y deja una salida utilizable.

El ejecutor persiste primero el script y el payload efectivo en disco, normalmente en rutas como `results/code/generated_<run_id>.py` y `results/logs/payload_<run_id>.json`. A continuación lanza el proceso Python con un tiempo máximo acotado, captura `stdout` y `stderr`, registra el código de retorno y guarda esas evidencias como artefactos observables. Esta persistencia es metodológicamente relevante porque permite reconstruir después no solo el script final, sino también el contexto exacto con el que se ejecutó y los mensajes que produjo el intento real.

La validación de esta fase se organiza alrededor de dos preguntas principales. La primera es si el script se ha ejecutado correctamente como proceso. Aquí se observa si el lanzador pudo iniciarlo, si la ejecución terminó dentro del tiempo permitido y si el `returncode` refleja un final correcto o un fallo explícito. La segunda pregunta es si la salida generada queda en una forma apropiada para que la siguiente fase pueda trabajar con ella. En este punto no basta con que el proceso haya terminado: hace falta además que `stdout` sea JSON parseable, que mantenga las claves estructurales esperadas —`metrics`, `summary` y `limitations`— y que no mezcle salida libre, trazas accidentales o formatos ambiguos.

Desde esta perspectiva, ejecutar no equivale automáticamente a validar. Un `returncode = 0` con un `stdout` vacío o con un JSON incompleto sigue siendo insuficiente para el workflow. Del mismo modo, una salida aparentemente bien formada puede seguir siendo inútil si no conserva las claves mínimas sobre las que debe apoyarse el interprete final. Por eso la arquitectura introduce aquí una decisión propia, también tipada en tres estados: `valid`, `repairable` y `blocked`. El caso es `valid` cuando el proceso termina correctamente y deja una salida estructurada reutilizable; es `repairable` cuando el fallo observado parece corregible mediante un nuevo script; y pasa a `blocked` cuando ya no existe una corrección razonable o se agotan los intentos permitidos.

Esta etapa da lugar al Subagente 4 cuando la ejecución falla de forma reparable. A diferencia del Subagente 3, que corrige problemas detectados antes de ejecutar, este nuevo subagente trabaja sobre evidencia real de runtime: recibe el script previo, el contexto operativo y un resumen compacto del error observado, incluyendo `returncode`, `stderr` y, si procede, la información sobre una salida mal formada. Sobre esa base devuelve un script completo corregido, que vuelve directamente al ejecutor para un nuevo intento. La lógica de control aquí es especialmente defendible: el Subagente 4 no certifica por sí mismo que el código ya esté bien; solo propone una reparación. La única confirmación fuerte de esa reparación es que el propio ejecutor consiga lanzar la nueva versión y obtenga de ella una salida `valid`.

En consecuencia, la parte 4 refuerza una idea central del trabajo: el código generado no se acepta por mera plausibilidad ni por una única comprobación previa, sino por el

hecho de dejar una evidencia tecnica satisfactoria cuando se enfrenta al contexto real del sistema. Esto mejora la explicabilidad interna del flujo, porque permite distinguir entre errores de planificacion, errores de implementacion y errores observados solo al ejecutar. Tambien mejora la trazabilidad metodologica, ya que cada intento deja un script persistido, un payload concreto, una salida observable y una decision clara sobre si el caso puede continuar, debe repararse o ya debe bloquearse.

5.5.5. Interpretacion y respuesta final

La ultima etapa del flujo corresponde al Agente 5, encargado de transformar una salida de ejecucion ya validada en una respuesta final comprensible para el usuario. Esta fase no recalcula metricas, no reabre la planificacion analitica y no vuelve a cuestionar la descarga realizada. Su funcion es distinta: tomar los resultados ya obtenidos, leerlos a la luz de la consulta original y redactar una respuesta fiel, prudente y ajustada a lo que realmente se ha conseguido producir.

La entrada del interpretador no coincide con todo el estado compartido. Antes de llamar al Agente 5, el workflow construye un `interpretation_payload` limpio. Ese paquete incluye la consulta original, el contexto resuelto minimo —por ejemplo, tickers y marco temporal—, la `execution_output` ya confirmada como valida y los avisos relevantes acumulados durante el flujo. Esta eleccion de diseno es importante porque protege al interpretador frente a ruido interno innecesario y, al mismo tiempo, mantiene la conexion explicita entre la parte 4 y la respuesta final.

En coherencia con la filosofia adoptada para esta fase, el payload del Agente 5 no debe incluir `analysis_plan` ni pistas equivalentes sobre el nivel de analisis esperado. Tampoco deberian llegarle etiquetas como **A**, **B** o **C**, preferencias internas de presentacion ni artefactos tecnicos como rutas de scripts, `stdout` completo o mensajes largos de infraestructura. La razon metodologica es clara: se quiere observar la capacidad del interpretador para decidir por si mismo cuanta elaboracion necesita la respuesta a partir de dos fuentes legitimas de informacion, la intencion expresada en la consulta y la riqueza real de los resultados ejecutados.

Esta separacion permite defender que el Agente 5 no actua como un mero reformateador ciego del `summary` producido por el script. Su tarea es mas sutil. Debe decidir si la respuesta conviene ser breve o desarrollada, si debe enfatizar una comparacion entre activos, si tiene sentido presentar bloques visibles o si basta una lectura sintetica. Esa decision no viene impuesta por una etiqueta externa, sino por la combinacion entre la consulta del usuario y el contenido factual de `execution_output`. Si la consulta es simple y la salida tambien lo es, la respuesta debe ser simple. Si la consulta pide comparacion, desglose o una lectura mas estructurada, el interpretador puede extenderse, pero sin fingir resultados adicionales ni sonar mas concluyente de lo que permiten los

datos.

Desde el punto de vista del control del flujo, esta etapa se apoya en una condicion previa fuerte: solo se alcanza cuando la parte 4 ya ha dejado una `execution_output` valida para el workflow. Eso significa que el interpretador no trabaja sobre un `returncode` dudoso ni sobre una salida rota. El sistema evita asi una degradacion frecuente en arquitecturas generativas mas simples, donde el modelo final acaba improvisando una respuesta “razonable” incluso cuando las etapas anteriores no han dejado una evidencia tecnica suficientemente solida. En este diseno, la interpretacion solo se produce cuando existe una base estructurada utilizable.

La salida principal de esta fase es `final_answer`, es decir, el texto final que vera el usuario. Sin embargo, la arquitectura conserva ademas una evidencia interna muy util: el propio `interpretation_payload` que vio el Agente 5. Esta decision mejora la trazabilidad de la ultima fase, porque permite reconstruir despues que hechos exactos se entregaron al interpretador y como se conectaron la validacion de ejecucion y la formulacion final de la respuesta. En la practica, esa informacion queda recogida dentro de la traza global del workflow y puede inspeccionarse en los snapshots finales de cada `run_id`.

Este bloque es tambien el mejor lugar para destacar la explicabilidad externa del sistema. No basta con que el flujo sea auditable por dentro; la respuesta final debe ser comprensible para un lector humano y debe distinguir con claridad entre hechos observados, lectura interpretativa y limitaciones relevantes. Cuando esta parte funciona bien, el sistema no solo “ha calculado algo”, sino que consigue cerrar el recorrido multiagente con una comunicacion clara, honesta y alineada con la peticion original del usuario.

5.6. Subagentes de correccion y recuperacion

Completar aqui el papel de los subagentes o mecanismos de reparacion. Conviene separarlos del flujo principal para que primero se entienda el recorrido base y despues se vea como reacciona el sistema cuando algo falla.

En este apartado deberias poder explicar cuando se activa cada correccion, que informacion recibe el subagente, que tipo de error intenta reparar y que criterio decide si el flujo continua o termina en error. Tambien interesa comentar si las reparaciones afectan al plan, al codigo, a la ejecucion o solo a una parte concreta del proceso.

Metodologicamente, esta seccion es importante porque muestra que el sistema no solo tiene una ruta ideal, sino tambien una estrategia controlada para gestionar fallos.

5.7. Diseno de prompts y contratos de los agentes

Este es el lugar adecuado para incluir los prompts dentro de la memoria principal. No conviene tratarlos como un anexo aislado, sino como parte del diseno operativo del sistema.

Completar aqui que objetivo tiene cada prompt, que entrada recibe cada agente, que salida se le exige y que restricciones guian su comportamiento. Puedes apoyarte en una tabla resumen y despues desarrollar los agentes mas importantes de forma narrativa.

Agente	Entrada principal	Salida esperada	Aspectos del prompt que conviene explicar
Agente de analisis	Consulta original mas payload compacto con tickers, contexto temporal, CSV normalizado y columnas disponibles	AnalysisPlan en JSON tipado	Objetivo analitico, catalogo de tipos de analisis, columnas disponibles, metricas requeridas y contrato de salida estructurada.
Agente de codigo	Consulta original, payload compacto y AnalysisPlan generado por el analista	JSON con un unico campo code que contiene el script completo	Contrato tecnico de lectura del payload, cargadores permitidos, imports autorizados, salida JSON unica y restricciones de implementacion.
Agente de interpretacion	Consulta original, contexto resuelto minimo, execution_output valida y avisos relevantes	Texto final para el usuario en final_answer	Uso exclusivo de resultados ejecutados, eliminacion de pistas internas, adaptacion de profundidad segun la query y prudencia frente a limitaciones reales.
Subagentes de correccion	<i>Completar</i>	<i>Completar</i>	<i>Tipo de error que reciben, alcance de la reparacion y limites de actuacion.</i>

Tabla 5.1: Guia para documentar los prompts y contratos de los agentes.

En este bloque debes poder defender no solo que existen prompts distintos, sino por que estan separados y que gana el sistema con esa especializacion. Si incluyes fragmentos textuales, conviene que sean los que mejor revelan el contrato de cada agente y no necesariamente el prompt completo sin explicacion.

5.8. Trazabilidad y artefactos generados

Uno de los rasgos mas importantes del sistema es que cada ejecucion deja una cadena de artefactos que permite reconstruir el flujo completo. Esta trazabilidad no depende

de logs narrativos genericos, sino de objetos y ficheros con una funcion concreta dentro de la arquitectura. Por eso puede afirmarse que la respuesta final no aparece como un resultado opaco, sino como la ultima pieza de una secuencia de evidencias observables.

En la transicion entre la fase de datos y la parte analitica, la primera evidencia relevante es el contexto resuelto que queda en `WorkflowState`. A partir de ese estado se construye el payload compacto que conecta la fase 1 con los agentes 2 y 3. Aunque ese payload no persiste inicialmente como fichero independiente, su contenido puede reconstruirse porque procede de campos tipados del estado, del resumen de descarga y de las rutas de los CSV normalizados.

El siguiente artefacto clave es el `AnalysisPlan`. Este objeto no se trata como una simple respuesta textual del LLM, sino como un contrato analitico formal que se parsea, se tipa y se conserva antes de pasar al codegen. Gracias a ello, el sistema mantiene una evidencia explicita de que calculos se querian realizar, con que columnas y con que formato de salida esperado. Desde el punto de vista metodologico, este punto es central: permite discutir por separado si el plan era adecuado y si el codigo lo implemento correctamente.

La segunda evidencia principal es el script generado por el Agente 3. Durante la parte 2 este codigo vive en memoria compartida como `generated_code`. Cuando la parte de ejecucion toma el relevo, el sistema lo persiste en `results/code/generated_<run_id>.py`. Junto a el se guarda tambien el payload efectivo de ejecucion en `results/logs/payload_<run_id>.json`, lo que permite revisar exactamente con que contexto operativo se lanzo el script.

Tras la ejecucion aparecen otros artefactos igualmente relevantes: `stdout`, `stderr`, el codigo de retorno y la salida parseada del script. Estos elementos se persisten en rutas como `results/logs/stdout_<run_id>.json` y `results/logs/stderr_<run_id>.log`, y se reflejan ademas en el estado compartido. Gracias a ello puede saberse no solo si el script produjo un resultado, sino tambien si fallo, por que fallo y si la salida respetaba o no el contrato esperado.

Finalmente, la lista de avisos, el mensaje de error y la respuesta final completan la trazabilidad funcional del sistema. Los avisos permiten registrar irregularidades no bloqueantes; el mensaje de error localiza la causa principal cuando el flujo se interrumpe; y la respuesta final muestra como se ha traducido la salida estructurada a lenguaje natural. En conjunto, estos artefactos permiten defender los resultados del trabajo con evidencias inspeccionables y no solo con una impresion subjetiva de que “el sistema parece responder bien”.

5.9. Decisiones de diseno

Completar aqui por que se elige este flujo y no otro. Este apartado debe servir para justificar las principales elecciones de arquitectura: separacion de responsabilidades, uso de varios agentes, validacion previa a la ejecucion, reparacion controlada, anclaje a datos locales y conservacion de artefactos.

No se trata de repetir la metodologia, sino de defender las decisiones tecnicas que hacen posible luego una evaluacion seria del sistema. En otras palabras, aqui explicas por que la arquitectura esta pensada para producir un analisis mas controlable, mas explicable y mas revisable que una respuesta directa generada de una sola vez por el modelo.

Capítulo 6

Resultados y evaluacion

6.1. Funcion de este capitulo

Este capitulo debe mostrar que ha ocurrido al ejecutar el sistema y como deben interpretarse esos resultados. Su objetivo no es solo presentar metricas, sino explicar que ensena la evaluacion sobre la robustez del flujo y sobre el valor real del trabajo.

6.2. Enfoque recomendado

Conviene dejar este capitulo planteado, no cerrado. Cuando llegue el momento, la redaccion final deberia combinar:

1. una lectura global del rendimiento;
2. un analisis por tipo de caso o por grado de exigencia de la consulta;
3. una lectura por etapas del flujo;
4. una discusion de fallos, reparaciones y limites.

6.3. Preguntas que convendra responder mas adelante

1. ¿Que significa exactamente que el sistema haya completado bien un caso?
2. ¿Donde se concentran los errores: datos, planificacion, generacion, validacion, ejecucion o interpretacion?
3. ¿Que gana el sistema con los subagentes de correccion?
4. ¿Que tipos de consulta tensan mas la arquitectura?

5. ¿Que parte de los resultados respalda de verdad la tesis del trabajo?

6.4. Plantilla de estructura futura

6.4.1. Resultado global

Completar aqui el resumen global de la evaluacion cuando el sistema este implementado y probado.

6.4.2. Lectura por grupos de casos

Completar aqui la segmentacion de consultas que resulte mas util: por tipologia, por numero de restricciones, por formato de salida exigido o por etapa tensionada del flujo.

6.4.3. Analisis por etapas

Completar aqui el rendimiento observado en cada bloque del flujo.

6.4.4. Casos representativos

Completar aqui varios ejemplos que ilustren un caso sencillo, uno intermedio, uno complejo resuelto y un fallo informativo.

6.4.5. Discusion critica

Completar aqui que se ha demostrado, que no se ha demostrado y que mejoras quedarian priorizadas para una siguiente iteracion.

Capítulo 7

Limitaciones, aspectos eticos y reproducibilidad

7.1. Funcion de este capitulo

Este capitulo reúne tres elementos que no son el centro del trabajo, pero si condicionan su interpretacion final: las limitaciones del estudio, el uso responsable del sistema y las condiciones para repetir el trabajo.

7.2. Limitaciones del estudio

Completar aqui limites de datos, dependencia del modelo, cobertura de casos, sensibilidad a prompts, dificultad de generalizacion y cualquier restriccion importante del sistema. Este es el lugar principal para reunir las limitaciones globales del trabajo, aunque en metodologia y resultados tambien puede ser util comentar limites especificos de cada bloque.

7.3. Aspectos eticos y uso responsable

Completar aqui finalidad prevista, usos excluidos, supervision humana, limites del analisis historico y riesgos de ejecutar codigo generado o de sobrerinterpretar resultados financieros.

7.4. Reproducibilidad

Completar aqui lo necesario para repetir el trabajo: entorno, dependencias, datos, artefactos conservados, variables de entorno, pasos minimos de ejecucion y factores que pueden hacer variar el resultado.

7.5. Preguntas de control

1. ¿Que no debería concluir un lector a partir del sistema?
2. ¿Que necesita exactamente otra persona para repetir un caso?
3. ¿Que parte del resultado depende del flujo y que parte depende del modelo concreto?
4. ¿Que riesgos siguen abiertos aunque el sistema funcione?

Capítulo 8

Conclusiones y trabajo futuro

8.1. Funcion de este capitulo

La conclusion final debe recoger la aportacion principal del trabajo con una formulacion clara y defendible. No conviene que sea una repeticion de capitulos anteriores ni una lista dispersa de ideas futuras.

8.2. Guia para las conclusiones

Completar aqui, al final del proyecto, una sintesis que responda a estas preguntas:

1. ¿Que problema has abordado realmente?
2. ¿Que has conseguido demostrar?
3. ¿Que evidencia es la mas importante para sostener esa afirmacion?
4. ¿Que limite obliga a interpretar el resultado con prudencia?

8.3. Guia para el trabajo futuro

Completar aqui las siguientes lineas de trabajo priorizadas: mejoras del flujo, robustez, validacion, ampliacion de casos, comparativas con otras soluciones y posible evolucion del sistema.

Bibliografía

- [1] Dogu Araci. Finbert: Financial sentiment analysis with pre-trained language models. *arXiv preprint arXiv:1908.10063*, 2019. doi: 10.48550/arXiv.1908.10063. URL <https://arxiv.org/abs/1908.10063>.
- [2] Ran Aroussi. yfinance python library, 2026. URL <https://github.com/ranaroussi/yfinance>.
- [3] Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. doi: 10.48550/arXiv.2107.03374. URL <https://arxiv.org/abs/2107.03374>.
- [4] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*, 2023. doi: 10.48550/arXiv.2304.05128. URL <https://arxiv.org/abs/2304.05128>.
- [5] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. *arXiv preprint arXiv:2211.10435*, 2022. doi: 10.48550/arXiv.2211.10435. URL <https://arxiv.org/abs/2211.10435>.
- [6] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 2022. doi: 10.1145/3571730. URL <https://arxiv.org/abs/2202.03629>.
- [7] Yuke Lai, Chengcheng Li, Yutao Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-tau Yih, Daniel Fried, Sida I. Wang, and Tao Yu. Ds-1000: A natural and reliable benchmark for data science code generation. *arXiv preprint arXiv:2211.11501*, 2022. doi: 10.48550/arXiv.2211.11501. URL <https://arxiv.org/abs/2211.11501>.
- [8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for

- knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2005.11401>.
- [9] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110*, 2022. doi: 10.48550/arXiv.2211.09110. URL <https://arxiv.org/abs/2211.09110>.
 - [10] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chengguang Zhu. G-eval: Nlg evaluation using gpt-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2511–2522. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.153. URL <https://aclanthology.org/2023.emnlp-main.153/>.
 - [11] OpenAI. gpt-oss-120b & gpt-oss-20b model card, 2025. URL <https://arxiv.org/abs/2508.10925>.
 - [12] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. Asleep at the keyboard? assessing the security of github copilot’s code contributions. In *2022 IEEE Symposium on Security and Privacy*, pages 754–768. IEEE, 2022. doi: 10.1109/SP46214.2022.9833571. URL <https://arxiv.org/abs/2108.09293>.
 - [13] Ji-Lun Peng, Sijia Cheng, Egil Diau, Yung-Yu Shih, Po-Heng Chen, Yen-Ting Lin, and Yun-Nung Chen. A survey of useful llm evaluation. *arXiv preprint arXiv:2406.00936*, 2024. doi: 10.48550/arXiv.2406.00936. URL <https://arxiv.org/abs/2406.00936>.
 - [14] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023. doi: 10.48550/arXiv.2302.04761. URL <https://arxiv.org/abs/2302.04761>.
 - [15] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *arXiv preprint arXiv:2308.11432*, 2023. doi: 10.48550/arXiv.2308.11432. URL <https://arxiv.org/abs/2308.11432>.
 - [16] Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhanjan Kambadur, David Rosenberg, and Gideon Mann. Bloomberggpt: A large language model for finance. *arXiv preprint arXiv:2303.17564*,

2023. doi: 10.48550/arXiv.2303.17564. URL <https://arxiv.org/abs/2303.17564>.
- [17] Yijia Xiao, Edward Sun, Di Luo, and Wei Wang. Tradingagents: Multi-agents llm financial trading framework. *arXiv preprint arXiv:2412.20138*, 2024. doi: 10.48550/arXiv.2412.20138. URL <https://arxiv.org/abs/2412.20138>.
- [18] Yahoo Finance Help. Download historical data in yahoo finance, 2026. URL <https://help.yahoo.com/kb/finance/historical-prices-adjusted-close-sln2311.html>.
- [19] Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang. Fingpt: Open-source financial large language models. *arXiv preprint arXiv:2306.06031*, 2023. doi: 10.48550/arXiv.2306.06031. URL <https://arxiv.org/abs/2306.06031>.
- [20] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- [21] Tianyu Zhou, Pingqiao Wang, Yilin Wu, and Hongyang Yang. Finrobot: Ai agent for equity research and valuation with large language models. *arXiv preprint arXiv:2411.08804*, 2024. doi: 10.48550/arXiv.2411.08804. URL <https://arxiv.org/abs/2411.08804>.

Apéndice A

Anexos

A.1. Ejecucion del sistema

Este anexo recoge la informacion operativa necesaria para ejecutar y defender el proyecto. El punto de entrada principal es `run_mvp.py`, que recibe una consulta en lenguaje natural, construye el workflow multiagente y devuelve un JSON final con el estado completo de la ejecucion.

A.1.1. Preparacion minima

Antes de lanzar una ejecucion conviene comprobar cuatro condiciones:

1. que el entorno Python tenga instaladas las dependencias del proyecto;
2. que el fichero `.env` contenga una configuracion LLM valida;
3. que el entorno tenga conectividad suficiente para acceder tanto al endpoint LLM como a `yfinance`;
4. que el proceso pueda escribir los artefactos generados en la carpeta `results`.

Una instalacion minima puede hacerse con:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

A.1.2. Ejecucion con ejemplos integrados

El proyecto incluye varios casos ya preparados en `src/examples/sample_inputs.py`. Estos ejemplos sirven para lanzar consultas representativas sin tener que redactarlas manualmente y permiten comprobar el flujo de extremo a extremo con peticiones ya

definidas dentro del repositorio. La forma mas simple de probar el sistema es ejecutar uno de esos ejemplos:

```
python run_mvp.py --example growth_nvda
python run_mvp.py --example returns_qqq_spy
python run_mvp.py --example complex_nvda_growth_report
```

Esta via es la mas util para una primera validacion, porque evita errores de formato en la entrada y permite observar como el sistema resuelve la peticion, descarga los datos necesarios y genera la respuesta final.

A.1.3. Ejecucion con una entrada JSON propia

Tambien es posible lanzar el sistema con un fichero JSON preparado manualmente:

```
python run_mvp.py --input-json ruta\al\caso.json
```

La estructura minima esperada es la siguiente:

```
{
  "query": "Analiza los retornos de QQQ y SPY en 2024"
}
```

La entrada minima solo necesita el campo **query**. A partir de esa peticion, el workflow resuelve los instrumentos, el rango temporal y los parametros de descarga antes de pasar a la fase analitica. La ejecucion normal del sistema no depende de proporcionar rutas de CSV locales en la entrada.

A.1.4. Salida y artefactos generados

La ejecucion imprime por consola un JSON con el estado final del workflow. Ese estado incluye, entre otros elementos:

- la consulta original y su forma normalizada;
- el plan analitico generado por el agente planificador;
- el codigo Python producido para el calculo;
- la salida estructurada de la ejecucion;
- la respuesta final redactada para el usuario;
- el estado de la ejecucion, los avisos y los posibles errores.

Ademas, el sistema conserva artefactos intermedios en dos carpetas:

- `results/data_raw` y `results/data_normalized`, con los artefactos generados durante la fase de descarga y normalizacion de datos;
- `results/code`, con el codigo generado en cada ejecucion;
- `results/logs`, con el payload enviado al ejecutor y los ficheros de `stdout` y `stderr`.

Esta conservacion resulta util tanto para depurar fallos como para defender metodologicamente la trazabilidad del flujo en la memoria.

A.2. Configuracion LLM y API

La aplicacion utiliza un endpoint compatible con OpenAI Chat Completions. El modelo `openai/gpt-oss-20b` [11] se sirve a traves de vLLM, aunque el codigo acepta varias familias de variables de entorno equivalentes.

A.2.1. Configuracion recomendada

La forma mas directa de configurar el sistema es declarar en `.env`:

```
LLM_PROVIDER=openai-compatible
VLLM_BASE_URL=https://servidor-vllm.example/v1
VLLM_API_KEY=pegar_aqui_la_clave
VLLM_MODEL=openai/gpt-oss-20b
```

A.2.2. Variables equivalentes admitidas

El codigo reconoce tres familias de nombres para base URL, clave y modelo:

- familia `VLLM_*`: `VLLM_BASE_URL`, `VLLM_API_KEY`, `VLLM_MODEL`;
- familia `LLM_*`: `LLM_BASE_URL`, `LLM_API_KEY`, `LLM_MODEL`;
- familia `OPENAI_*`: `OPENAI_BASE_URL`, `OPENAI_API_KEY`, `OPENAI_MODEL`.

La resolucion de configuracion prioriza primero los valores `VLLM_*`, despues los valores genericos `LLM_*` y finalmente los alias `OPENAI_*`.

A.2.3. Parametros de generacion

Tambien pueden ajustarse los parametros principales del cliente:

```
LLM_TEMPERATURE=0.1  
LLM_MAX_TOKENS=4096  
LLM_TIMEOUT_SECONDS=120  
OPENAI_VERIFY_SSL=true
```

Estos valores controlan la temperatura de generacion, el maximo de tokens de salida, el tiempo de espera del cliente y la verificacion SSL del endpoint remoto.

A.2.4. Comprobaciones practicas

Si la configuracion es correcta, una ejecucion con `run_mvp.py` debe completar el flujo resolviendo la consulta, descargando los datos historicos necesarios y generando despues una respuesta trazable. Si la clave no esta definida o contiene un valor de ejemplo, el sistema no intenta llamar al modelo y devuelve un error de configuracion explicito.

A.3. Notas para reproducibilidad

Este anexo sirve como referencia corta de reproduccion operativa. Para repetir un caso hacen falta el mismo codigo, una configuracion equivalente del modelo, acceso a un endpoint compatible con OpenAI y disponibilidad de los datos historicos correspondientes en `yfinance`. Si alguno de estos elementos cambia, el comportamiento del sistema puede variar aunque la arquitectura general siga siendo la misma.